AutoDoc AI

Technical Report

Automotive Document Assistant

Program	TechPulse — Applied AI/ML Capstone
Prepared by	Kaavika Sri S
Department	B.Tech CSE (AI & DS)
Institution / Register No.	SASTRA Deemed University 127156065

Abstract

AutoDoc AI is a small web application for searching automotive engineering PDFs. It extracts text from an uploaded file, prepares searchable passages, ranks passages against a user question, and shows source pages and similarity scores. The user can check the results in the original document. The current prototype does not use a generative LLM and does not approve engineering decisions.

1. Problem and Intended Users

High-Level Design (HLD)documentscan contain information about software components, interfaces, signals, dependencies and functional flows over many pages. Finding a particular detail manually can take time. AutoDoc AI aims to help users find possible evidence faster. Intended users include system and AUTOSAR architects, developers, integration engineers and test engineers.

2. Objectives

- Upload anautomotive engineering PDF through a browser interface.
- Extract available text and retain page information.
- Prepare passages that can be searched.
- Rank passages against a question using text similarity.
- Display the retrieved text, page number and similarity score for manual checking.

3. Input Data

Thedemo used Automotive_Engineering_AI_Project_Case_Studies.pdf, a case-study document that includes an AUTOSAR HLD Document Analysis Assistant use case. The app reported 56 searchable passages, 26 pages with extracted text and an upload size of about 0.55 MB. These are observations from this file only, not a general performance benchmark.

4. Workflow and Architecture

1. Upload PDF
2. Extract text and page details with PyMuPDF
3. Prepare searchable passages
4. Rank passages using TF-IDF text similarity
5. Show passages, page references and scores
6. User checks the original PDF

This describes the implemented retrieval-based prototype. It is not a full RAG pipeline: a generative LLM and vector database are not included in the current version.

5. Tools and Configuration

Tool	Use
Python	Application logic and text handling.
Streamlit	Browser interface for upload, query and results.
PyMuPDF (fitz)	PDF text extraction and page information.
TF-IDF text similarity	Ranks passages against the query.
Anaconda	Python environment used to install and run the app.

The demonstrated setup used a Conda environment named autodoc. From the folder containing app.py and requirements.txt, dependencies were installed with python -m pip install -r requirements.txt, and the app was started with streamlit run app.py. It was accessed locally at http://localhost:8501.

6. Evaluation and Observed Results

A manual test loaded the case-study PDF and used the question “What are the objectives of the AUTOSAR HLD Document Analysis Assistant?” The results included a useful passage on page 4 about reducing manual review effort and making architecture knowledge searchable. Metadata on page 2 also ranked highly, so the search still needs improvement.

Test	Observed result	Status
PDF upload	PDF loaded and passages prepared.	Observed working
Document statistics	56 passages; 26 pages with text.	Observed working
Question search	Ranked passages with page numbers and scores.	Working; relevance imperfect
Formal accuracy benchmark	No labelled benchmark was run.	Not completed

Similarity scores are ranking values. They are not confidence scores or proof that a result is correct.

7. Responsible Use

- Source passages and page numbers let users verify findings against the PDF.
- A human must review results; the app does not approve architecture or safety decisions.
- Use only documents that you are allowed to process and share.
- Login, role-based access, audit logging and production security are not implemented.
- Describe the app as text retrieval, not as a generative LLM or full RAG system.

8. Project Contribution

The prototype puts PDF text extraction, question-based ranking and page references in one simple interface for automotive documents. Its contribution is helping users locate candidate evidence. It does not include custom agents, generated answers, prompt pipelines or persistent vector storage.

9. Limits and Future Work

- Some metadata or repeated headers may rank above the most useful passage.
- The app retrieves passages but does not write a natural-language answer.
- Scanned PDFs and complex layouts may need additional extraction methods.
- Testing was limited to one demonstration PDF and a small number of questions.
- Future work could improve text cleanup, evaluate a labelled question set, add semantic embeddings, or optionally add an approved LLM that cites retrieved evidence.

10. Conclusion

AutoDoc AI demonstrates a working first step for searching automotive engineering PDFs. It extracts text, ranks passages against a question and points users to source pages. The next step should be to measure and improve retrieval relevance before adding more advanced AI features.

11. Source and AI-Tool Disclosure

The demonstration input and project context came from Automotive_Engineering_AI_Project_Case_Studies.pdf. Cite this source in the format required by the program. ChatGPT assisted with planning, code drafting/refinement, troubleshooting, presentation preparation and documentation drafting.